

Diagnosing Memory-Write Decisions in Conversational Assistants Through Specification-Based Replay

ANONYMOUS AUTHOR(S)

Conversational assistants with persistent memory must complete requested corrections without corrupting stored facts. A final-state score cannot show whether a difference between memory-write rules comes from the rule, an earlier stage, the reference's matching of written forms, or a lack of cases on which the rules could differ. Specification-based replay holds generated extractions and proposals fixed, varies only the admission rule, scores whole executed states, and counts exposure, the outputs on which rules can differ; exposure bounds any difference between two rules and confines the reference's influence on it to those outputs. On 48 Korean test cases and 21 model configurations, replay separates three confounds. A correction gain credited to evidence checks was the release of holds created by extraction, which a revised extraction instruction removed; most ties reflected zero exposure; and whether a gate on general-review proposals appears protective depends on the reference: accepting surface variants of required facts cuts the violations it avoids by almost two thirds and more than doubles its cost in completed changes, while comparisons among rules on fixed proposals are unchanged. Replay also explains a prespecified null result and, on real speech-recognition hypotheses, traces holds to genuine disagreement. An audit of ten memory-write evaluations shows that seven assess writes only through downstream answers.

CCS Concepts: • **Computing methodologies** → **Discourse, dialogue and pragmatics**; *Nonmonotonic, default reasoning and belief revision*; *Intelligent agents*; • **General and reference** → Evaluation.

Additional Key Words and Phrases: conversational agents, persistent memory, memory correction, admission control, specification-based evaluation, failure diagnosis

ACM Reference Format:

Anonymous Author(s). 2026. Diagnosing Memory-Write Decisions in Conversational Assistants Through Specification-Based Replay. *ACM Trans. Intell. Syst. Technol.* 1, 1 (October 2026), 22 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Conversational assistants built on large language models increasingly keep a persistent memory of their users [11]. Updating this memory requires both completing requested corrections and preserving facts that should remain unchanged. Current systems decide whether to add, revise, or delete stored facts [4, 7, 23, 41, 42] and are usually evaluated later through questions about long conversations [22, 39]. Such a downstream or final-state score summarizes an outcome, but it does not identify which proposed write, admission decision, or reference convention produced it. This article studies that diagnostic gap.

A simple example illustrates the problem. Suppose that the stored opening year of a shop is 1991 and the reference requires a correction to 1993. The model proposes the correction, yet the application keeps 1991 because an agreement check does not find the corrected tuple in the facts extracted from every reading of the utterance. A rule that releases the hold completes the correction, but whether the credit belongs to that rule or to the extractor that omitted the tuple cannot be read from the final state.

We therefore separate generation, which produces a proposal; admission, which decides which proposed operations may execute; and execution, which changes the persistent state. Holding generation fixed and varying only admission makes every difference in the resulting state attributable to admission within the tested pipeline. Four rules then act on identical operations. AGREE admits

a correction only when its tuple is shared by the extracted facts of all readings, and ALLOW admits every correction that AGREE holds for lack of shared support. Two evidence checks admit such a held correction only when its new value (LITERAL) or an affirmed quotation linked to the same target record (WITNESS) occurs in every reading. Because the rules can differ only on outputs that contain a held correction, we count these outputs and call the count *exposure*.

Replay separates three confounds, each a pair of situations that a final-state score cannot distinguish (Figure 1). In the *attribution* confound, a difference credited to the admission rule arises in another stage. In the *specification* confound, an avoided reference violation reflects how the reference matches written forms rather than a semantic error. In the *opportunity* confound, a tie reflects that no proposal reached the rules' decision boundary. Each has a precedent in state tracking: oracle analysis attributes errors to stages [15], and benchmark revisions list the surface variants of a value as equally correct [46]. What is missing is a way to separate them for memory writes, which are judged on the whole executed state. Of ten memory-write evaluations that we audited, seven assess writes only through downstream answers, and none reports how many cases could distinguish the compared policies (Table 1).

Three research questions organize the study. RQ1 asks where admission can intervene and what produces the holds that it releases (Sections 6.1 and 6.2). RQ2 asks how evidence requirements change completion and reference violations on fixed proposals and how much these comparisons depend on the reference (Sections 6.3 and 6.4). RQ3 asks why a prespecified complete-pipeline test found no completion advantage and what changes with a learned verifier and with real recognition hypotheses (Sections 6.5–6.7).

This article makes three contributions:

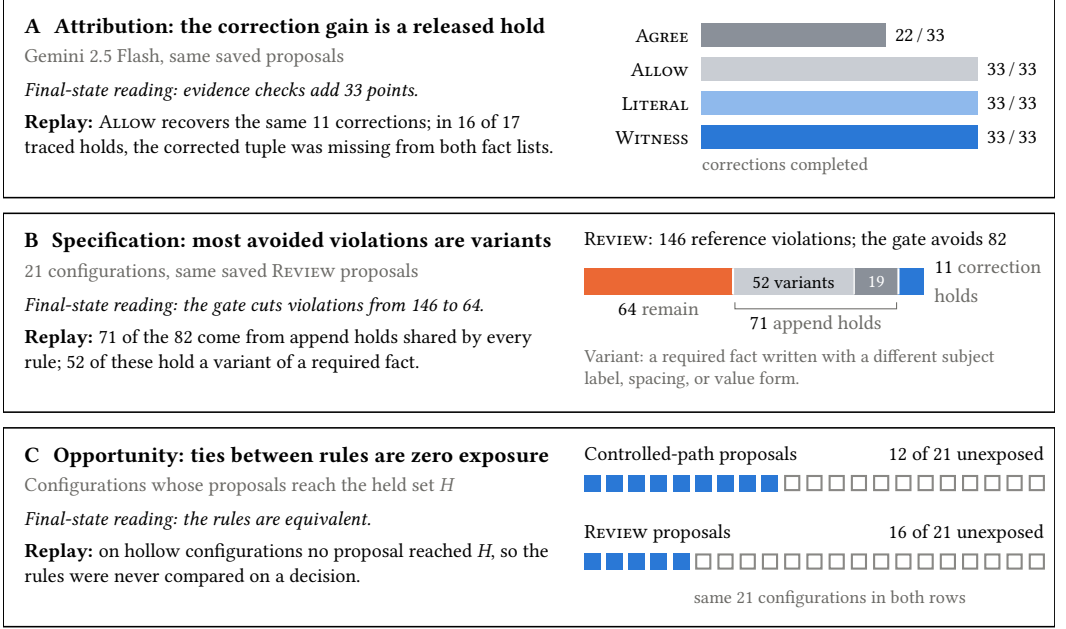
- *Specification-based replay with exposure.* The protocol extends fixed-candidate replay [17] to edits of identified records under competing readings, scores whole executed states, and counts exposure. Exposure bounds any difference between two rules on the same proposals and confines the reference's influence on that difference to the exposed outputs (Proposition 4.1). The protocol runs on saved artifacts: across 21 configurations and 3,024 episodes, a re-executed gate reproduces every saved score, and the same machinery replays a learned verifier and recognizer hypotheses (Sections 4–6).
- *Three confounds, each traced to a cause and tested.* Extraction, not the evidence check, produced the held corrections that the checks were credited with releasing, and a revised extraction instruction removed them; zero exposure explained most ties; and the reference decided whether a gate looked protective (Sections 6.1–6.4). On real recognition hypotheses, the same trace returned a different cause, disagreement between hypotheses (Section 6.7).
- *A robustness analysis of the reference.* Re-scoring every episode under a reference that accepts surface variants of required facts separates the conclusions that hold under both references, namely all comparisons among rules on fixed proposals, from those that do not: the apparent safety of a gate on general-review proposals and the direction of the complete-pipeline comparison (Sections 6.4 and 6.5).

Scope of claims. The claims are existence and attribution claims, established by traced artifacts whose counts are exact and reproducible; how often each confound arises in other systems is a separate question (Section 8).

2 RELATED WORK

2.1 Memory Updates and Admission in LLM Agents

Several systems give LLM agents a persistent memory that is updated through explicit operations. MemGPT treats the context window as fast memory and moves information to and from external



Counts pool three repetitions. Reference violations follow the frozen state specification.

Fig. 1. Three evaluation confounds that a final-state score cannot distinguish: attribution (A), specification (B), and opportunity (C). Each row sets what the final-state score shows beside what replay of the same saved artifacts shows. A: Gemini 2.5 Flash, strict corrections completed out of 33 observations under each rule. B: The 146 reference-violating states under REVIEW on the 21-configuration panel, split by whether the gate avoids them and, if so, by the held operation; “variants” are append holds whose tuple differs from a required fact only in subject label, spacing, or value form (Section 6.4). C: The 21 panel configurations, with controlled-path and with REVIEW proposals; hollow squares mark configurations on which no proposal reached the held set *H*.

storage through function calls [23]. Mem0 extracts candidate facts and applies one of four operations to existing memories: add, update, delete, or no operation [7]. A-Mem stores memories as linked notes whose attributes are revised as new notes arrive [41], and MemoryBank, recursive summarization, SeCom, LD-Agent, and Generative Agents maintain or select conversational history in related ways [16, 24, 25, 37, 51]. Closest to the correction decision studied here, Bae et al. cast memory management in Korean long-term conversation as pairwise operations between a stored sentence and a new session summary (pass, replace, append, or delete) [4]. They compared trained classifiers with natural language inference models mapped to these operations and evaluated operation accuracy, set-level overlap, and human ratings of later sessions. Memory-R1 learns add, update, delete, and no-operation decisions from a reward based on downstream answer correctness [42], and Zep invalidates, rather than deletes, knowledge-graph edges that newer information contradicts [28].

A recent line of work treats the write itself as an admission decision. A-MAC learns an admission policy from interpretable factors (future utility, factual confidence, semantic novelty, temporal recency, and a content-type prior) and evaluates admission decisions against labeled candidate memories [47]; ConsistencyGate samples the same model several times to score whether a candidate fact is supported by its source and admits the fact above a threshold [49]; and DerivAudit

audits whether a stored memory follows from the history available when it was written, finding that support often lies beyond the writer’s citations and that verifiers still admit many unsupported memories [17]. STALE evaluates whether agents notice that a stored memory has been implicitly invalidated by later information [5]. Memory writes are also an attack surface: Agent-Poison and MINJA show that poisoned or injected records, the latter inserted through ordinary queries, later steer agent behavior [6, 8]. In our setting, the surrounding history is supplied rather than retrieved, so retrieval failure is not confounded with admission failure; how the position of information affects its use is a separate question [18].

2.2 State Revision and Competing Hypotheses

Dialogue-state tracking already models state revision. SOM-DST predicts an operation for each slot before generating values [15], and its oracle analysis attributes most errors to the operation predictor rather than to value generation, an early stage-level diagnosis of the kind we apply to memory writes. AG-DST amends an initially generated state [35], dataflow synthesis supports executable dialogue programs and non-destructive revision [3], and PoA generates semantic-action code that updates the dialogue state [33]. Other trackers select the relevant turns, suppress incorrect values, or rewrite queries to resolve coreference [12, 19, 36]. The second Dialog State Tracking Challenge supplied competing speech-recognition hypotheses for every turn and scored the tracked state turn by turn [13]; its setting is close to ours, although there the tracker produces the state itself, and Section 6.7 replays our admission rules on its hypotheses. MultiWOZ 2.2 removes values that the system offered before the user accepted them [46], and MultiWOZ 2.4 removes values that only the system mentioned [44]; both revisions show why the timing and source of evidence matter. MultiWOZ 2.2 also lists the surface variants of a value as equally correct [46], a reminder that an exact-match reference can count an equivalent mention as an error (Section 6.4 returns to this issue). Our comparison intervenes after extraction and proposal are fixed: it evaluates the permission to execute a proposal and thus complements these state-generation methods.

2.3 Evidence, Clarification, and Evaluation

Clarification has long been modeled as a decision under uncertainty [9, 10, 26, 45, 48], although human clarification is only weakly tied to annotated ambiguity [21] and is not mirrored by model uncertainty [34]; an uncertain detail can be unnecessary for the next action yet consequential for a durable write. Disagreeing readings resemble the inter-context conflicts studied between retrieved sources [40], and a quoted span shows that a value occurs in the source, not that it supports the proposed fact [1]; this distinction motivates comparing literal evidence with target-linked witnesses. Attribution to identified sources can be defined and rated by people [27], entailment-based checkers detect factual inconsistency [14], LLM judges can approximate human preferences [50], and agreement across readings resembles self-consistency over sampled outputs [38]. τ -bench scores agents by the final database state after tool use [43], the kind of executed-state endpoint that we apply to a single turn. Self-Refine and Reflexion revise outputs at inference time [20, 31]; we use a two-call general review as the alternative complete pipeline, and because the choice of instruction can itself change results [2], every condition receives the same task facts and retention constraints. Automatic and human assessments can agree [32] or rank systems differently [30], so agreement has to be established for each endpoint. Table 1 audits what memory-write evaluations report.

DerivAudit compares verification settings on the same candidate memories and reports both supported-memory retention and unsupported admission [17], so replaying fixed candidates is not in itself our contribution. We add edits of identified records under competing readings, whole-state scoring, exposure with its bound, and an analysis of how far each conclusion depends on the

Table 1. Reporting audit of ten memory-write evaluations, from what each paper reports. *Fixed*: the compared write or admission policies act on the same generated candidates. *Both errors*: missed required writes and wrongly admitted writes are reported, not only downstream answers. *Opportunity*: the number of cases on which the compared policies could, or did, differ. *Matching*: how outputs are compared with references; n.r., not reported.

Work	Write decision scored	Fixed	Both errors	Opportunity	Matching
MemoryBank [51]	Downstream answers	–	–	–	Human ratings; agreement n.r.
MemGPT [23]	Downstream answers	–	–	–	LLM judge and ROUGE-L; agreement n.r.
Mem0 [7]	Downstream answers	–	–	–	Token overlap and LLM judge; agreement n.r.
A-Mem [41]	Downstream answers	–	–	–	Token and embedding overlap
Zep [28]	Downstream answers	–	–	–	LLM judge
Memory-R1 [42]	Downstream answers	–	–	–	Token overlap and LLM judge
STALE [5]	Downstream probes	–	Partial	–	LLM judge with human check
Bae et al. [4]	Operation per memory pair	Yes	Partial	–	Human labels, majority of three
A-MAC [47]	Admission per candidate	Yes	Yes	–	Candidate labels; process n.r.
ConsistencyGate [49]	Admission per fact	Partial	Yes	–	Constructed labels
This work	Admission, whole state	Yes	Yes	Yes	Exact and variant-tolerant reference; no human agreement

reference. We do not compare directly with A-MAC or ConsistencyGate, which score the admission of candidate facts against labels rather than whole executed states; Section 6.6 instead adds a learned verifier as a further rule on the same proposals.

3 TASK AND DECISION BOUNDARY

A test case is $x = (h, T, M, \pi)$, consisting of the preceding context h ; an indexed sequence $T = (t_1, \dots, t_k)$ of alternative readings of one current utterance, which may coincide; an initial memory state M ; and a case-level retention permission π . The readings are simultaneous alternatives for the same speaker's turn, not separate speakers or successive turns. A record has a persistent identifier and a fact tuple (subject, relation, value, time). The model proposes an action a , a reply u , and an ordered list of memory operations Q ; the application executes the admitted operations in one transaction and returns the state M' together with an execution log ρ .

Four operations are available: APPEND, CORRECT, HOLD, and no-write. A correction targets an existing record and replaces its value while keeping its identifier, whereas a hold leaves a proposed fact uncommitted. Retention permission is separate from factual support, because a clearly stated fact may still be disallowed for storage. Supplying the initial state lets the evaluation distinguish a new fact from an obsolete or protected one without having to infer it. For each case, the reference specifies the allowed actions, the required and forbidden tuples, the records that must remain unchanged, the obsolete versions to retire, and any records that must no longer appear; all of these requirements are checked on the executed state.

We assess required change and preservation together. On a positive case, at least one required fact is absent or an obsolete version must be retired, so holding every proposal fails. A correct target update can still fail whole-state conformity when it also appends a fact outside the requested

Table 2. Terms and endpoints. With three repetitions, the correction, positive, and all-case denominators are 33, 84, and 144 per configuration; with one repetition, they are 11, 28, and 48.

Term	Definition
Reading	One written alternative of the current utterance, supplied as text.
Extraction F	Reading-level fact lists and correction witnesses produced by the first model call.
Proposal (a, u, Q)	Action, reply, and memory operations produced by the second model call.
Held set H	CORRECT operations that AGREE holds only for lack of shared support, although the extraction validly covers every reading.
Exposure	Outputs on which the rules can decide differently, that is, outputs that contain an operation in H , reported with the number of distinct cases they represent.
Complete pipeline	A system that generates its own proposal. REVIEW: a draft plus one general-review call.
Controlled path	Extraction, proposal, and WITNESS admission; reply rendered from the execution log.
Strict correction completion	Allowed action and exactly the reference-specified whole state on the 11 correction cases.
Positive-change completion	Allowed action and exact whole state on the 28 positive cases (the prespecified primary endpoint).
Reference-violating state	Forbidden or extra write, or damage to a protected fact, as defined by the reference (all 48 cases).
Whole-state conformity	Executed state meeting every state requirement of the reference (all 48 cases).
Negative preservation	Preservation of the state on the 20 cases that require no change.

edit. We score the action and persistent state against the reference and inspect reply–state discrepancies in selected traces; reply truthfulness is not a scored endpoint. In tables, “Viol.” abbreviates reference violations. Table 2 defines the terms and endpoints used throughout.

4 CORRECTION ADMISSION RULES

4.1 One Proposal and Four Admission Decisions

The controlled path makes two model calls. The first returns an extraction F : for each reading, the atomic facts that the reading supports, and, for each pair of a stored record and a reading, one correction witness with a polarity label (affirmed, negated, uncertain, or absent), a new value, and a verbatim quotation. The second call receives the test case together with F , marked as fallible, and generates the final proposal. Every admission rule receives the identical test case, extraction, and proposal, together with an isolated copy of M , so changing the rule requires no further model call (Figure 2).

AGREE applies to every proposed APPEND and CORRECT operation. It admits an operation when the exact subject–relation–value–time tuple is supported by the fact list of every reading or is already present in M ; otherwise, it changes the operation to HOLD. AGREE was the conservative gate of the original protocol, fixed before collection, and is meant to keep a fact out of memory when the readings disagree about it. Shared support requires that the extraction validly cover all readings. Because the test compares tuples exactly as extracted, and because the extraction schema allows a reading’s fact list to be empty, a correction can appear in the proposal and in the witnesses yet be missing from the intersection of the extracted fact lists. This is how the held set arises.

Table 3. Admission rules applied to the same proposal. Outside H , every rule keeps AGREE's decision; retention permission and execution checks are shared.

Rule	Admission within H
AGREE	Keeps the conservative hold when shared extracted support is missing.
ALLOW	Admits every otherwise eligible correction in H .
LITERAL	Checks the existing target and the unchanged subject, relation, and time; requires the new value in every reading and current-turn evidence.
WITNESS	Additionally requires one affirmed, matching, target-linked witness per reading, supported by an exact source quotation that contains the value.

Let $H(x, F, Q)$ be the set of operation indices j for which AGREE holds a proposed CORRECT only because its support is not shared, although every reading is validly covered. All differences between AGREE and the other three rules lie within H ; outside H , every rule keeps AGREE's decision (Table 3). ALLOW restores every correction in H and nothing else, so an APPEND that lacks shared support stays held under every rule. Two predicates define the evidence checks. For LITERAL ($L_j = 1$), the proposal must name one existing target; preserve its subject, relation, and time; change its value; and cite the current turn in both the fact and the operation. In addition, the replacement value must occur literally in every reading. For WITNESS ($W_j = 1$), the same target and source conditions must hold, and each reading must supply exactly one witness for the target record that is labeled affirmed, repeats the proposed subject, relation, time, and value, cites the current turn, and quotes a verbatim span of that reading containing the replacement value. The application verifies the structure and linkage of these model-generated witnesses, but the polarity label itself is taken from the model.

Two examples show how the checks differ. The sentence “the shop opened in 1993, not 1991” satisfies both checks for an opening-year record. By contrast, “My son works at Daehan Post” satisfies the literal check for an update of the speaker's own workplace but should fail the witness check, which requires every reading to affirm the value for the targeted record: the affirmed statement concerns the son.

4.2 Containment and the Opportunity to Intervene

The admitted sets of the four rules are nested. Write A_X for the indices in H that rule X admits; then

$$A_{\text{WITNESS}} = \{j \in H : W_j = 1\} \subseteq A_{\text{LITERAL}} = \{j \in H : L_j = 1\} \subseteq A_{\text{ALLOW}} = H. \quad (1)$$

The first inclusion holds because a passing witness carries the required target and source linkage and an exact quotation that contains the value, so the value occurs in every reading; the second follows from the restriction to H . This ordering concerns admitted operations only. It need not carry over to final-state accuracy when a transaction is rejected, when operations interact, or when a witness is wrong.

AGREE and ALLOW are therefore the two ends of this ordering rather than competing designs: no rule that acts on H can admit more held corrections than ALLOW or fewer than AGREE. We compare LITERAL and WITNESS with both ends: with ALLOW for the reference violations they prevent, and with AGREE for the corrections they recover and the reference violations they add. Because AGREE reads model-extracted facts, its holds can originate in extraction rather than in disagreement between readings; the saved extraction shows which of the two occurred (Section 6.1). WITNESS and LITERAL decide differently only on $K = \{j \in H : L_j = 1, W_j = 0\}$, where a hold may either

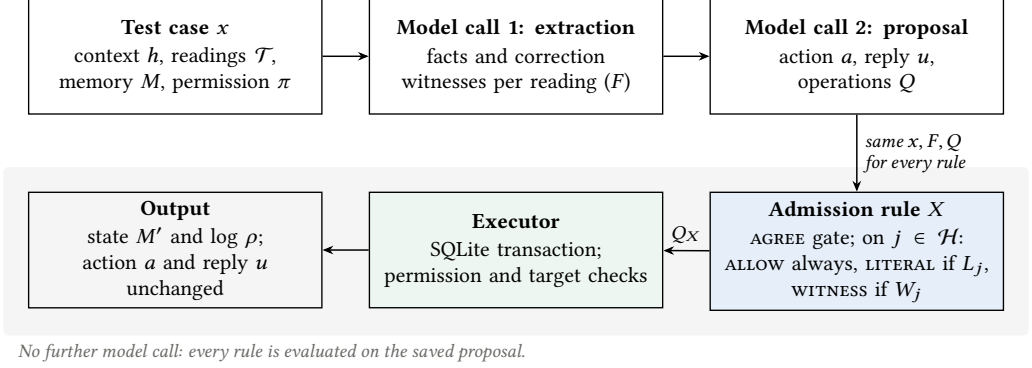


Fig. 2. Fixed-proposal architecture. Two generation calls produce one extraction and one proposal. Each admission rule is then applied to the same artifacts and executed on an isolated copy of the initial state.

protect the correct state or keep an obsolete fact unnecessarily. When H is empty, all four rules must agree. The following bound makes this precise for any two rules and any endpoint.

PROPOSITION 4.1 (EXPOSURE BOUND AND REFERENCE DEPENDENCE). *Let rules X and Y act on the same N outputs, and let E be the set of outputs that contain an operation on which X and Y can decide differently. For every endpoint indicator S computed on the executed state, (i) $|\bar{S}_X - \bar{S}_Y| \leq |E|/N$, and (ii) the difference $\bar{S}_X - \bar{S}_Y$ depends on the reference only through the scores of the outputs in E .*

PROOF. Outside E , the two rules admit the same operations, so the deterministic executor produces the same state and S takes the same value under X and Y . Hence $N(\bar{S}_X - \bar{S}_Y) = \sum_{o \in E} (S_X(o) - S_Y(o))$, which is at most $|E|$ in absolute value and involves the reference only through outputs in E . $\square$

For the four rules, E is the set of outputs that reach H ; for a gate applied to proposals that were not gated before, E contains every output with a held APPEND or CORRECT or an invalid extraction. Exposure therefore sets the resolution of a comparison: with 11 exposed outputs among 144, no rule comparison can exceed 7.6 percentage points, and with none, a tie carries no information about the rules. Part (ii) predicts which comparisons a change of reference can move, which Section 6.4 tests.

4.3 Execution and Complete Pipelines

The application applies operations transactionally to a separate SQLite state for each rule and records read-after-write execution logs; the executor enforces retention permission and target existence for every rule. Because admission does not regenerate the action or the reply, differences in state outcomes between rules can be assigned to admission alone within this pipeline. For the same reason, a held write can coexist with a reply that asserts the held value (Online Appendix H), and the unchanged reply is not credited to admission. WITNESS uses the same two calls per output as AGREE, and ALLOW and LITERAL are computed from the saved proposals (Algorithm 1; Online Appendix C).

The comparison pipeline, REVIEW, generates its own proposal from a draft and one general-review call, so it and the controlled path use the same number of calls. The original protocol also ran a single direct draft and one draft-feedback-refine cycle adapted from Self-Refine [20]; they share REVIEW's initial draft where the design permits and are reported in Online Appendices A

Algorithm 1 Admission on a fixed proposal. Reply rendering is a separate downstream option.

Require: case $x = (h, T, M, \pi)$, extraction F , proposal (a, u, Q) , rule X (AGREE, ALLOW, LITERAL, or WITNESS)

Ensure: action a and reply u unchanged; executed state M' and log ρ

```

1: if  $Q$  fails schema validation then
2:   return  $(a, u, M, \emptyset)$  ▷ no write; scored as an invalid output
3: end if
4:  $Q_X \leftarrow \text{AGREEGATE}(x, F, Q)$  ▷ holds APPEND and CORRECT without shared support
5: for all  $j \in H(x, F, Q)$  do
6:   if  $X = \text{ALLOW}$  or  $(X = \text{LITERAL and } L_j = 1)$  or  $(X = \text{WITNESS and } W_j = 1)$  then
7:      $Q_{X,j} \leftarrow Q_j$  ▷ restore the proposed correction
8:   end if
9: end for
10:  $(M', \rho) \leftarrow \text{EXECUTE}(M, Q_X, \pi)$ 
11: return  $(a, u, M', \rho)$ 

```

and F . The prespecified primary test compares the controlled path, with its reply rendered from the execution log, with REVIEW. Rendering does not change any state endpoint, so the tables report the common state score under WITNESS (Online Appendix A).

5 EVALUATION DESIGN

5.1 Sources and Model Configurations

The main source contains 60 Korean test cases written by the authors with AI assistance: 12 development cases and 48 held-out cases spanning 12 problem families of four cases each (Online Appendix A, Table A3). Each held-out case was run three times per model configuration. A private reference, fixed before the main collection, specifies the requirements listed in Section 3, including the distinction between replacing a value and removing its record. The collection script received the public cases but not the reference, and the configurations, prompts, admission rules, and evaluator were fixed in a frozen protocol before collection. We use Korean because widely used memory benchmarks are in English [22, 39] and because Korean writing yields several surface forms of one fact (word spacing varies in practice, and a person may be named by a kinship term, a given name, or both), which stresses exact tuple matching (Section 6.4).

All alternative readings of the main source are written text authored with AI assistance. Within a case, they can be identical, differ in surface form, or differ in value, person, or polarity; they play the role of competing recognition hypotheses but were not produced by a recognizer, so how often they reach the held set characterizes this source rather than any recognizer.

The original protocol planned a Holm-adjusted primary comparison for five model families; Gemini 2.5 Flash, Llama 3.1 8B, and Qwen3 14B (local 4-bit) were collected, and two families were not (Online Appendix C). Six later Gemini configurations reuse the 48 cases and were added after the earlier results had been inspected. The model panel comprises the 21 configurations that completed all 48 cases in all three repetitions with at least 95% valid final outputs in both REVIEW and the controlled path: seven Gemini, eight Claude, and three GPT configurations together with DeepSeek 3.2, GLM 5, and Qwen3 14B, for 3,024 case-repetition episodes. This validity criterion was set after collection and applies to complete pipelines, not semantic outcomes; of the 28 configurations with complete triplets, seven fall below it, among them Llama 3.1 8B (Online

Table 4. Sources and their roles. Additional configurations reuse the main cases, and each source keeps its own denominators.

Source	Cases	Repetitions	Role
Main Korean cases	48	3	Fixed-proposal and complete-pipeline comparisons
DSTC2 test turns	48	1	Real recognition hypotheses; 41 callers
Contrast set	24	1	12 supported and 12 unsupported edits
Boundary probe	12	1	Six paired edits; no exposure
Public dialogue probe	16	1	10 eligible self-repairs; six unscored
Follow-up runs on main cases	5; 48	1	Revised extraction; learned verifier

Appendix G). Model labels denote recorded provider or runtime configurations and do not imply matched parameter counts, precision, or reasoning budgets.

Table 4 lists the further sources. Three authored or adapted sources probe transfer and are reported in Online Appendices D–F: a 24-case contrast set written after the initial finding by a separate agent that saw neither the rule code nor earlier outputs, a 12-case boundary probe, and 16 selected MultiWOZ 2.4 turns [44]. A fourth source, 48 user turns from the DSTC2 test set, supplies real speech-recognition hypotheses with a reference derived from human-reviewed labels (Section 6.7). Three follow-up runs, collected after the results above had been inspected and fixed before their outputs, test the diagnoses (Online Appendix J): a revised extraction instruction on the held cases, a learned verifier (Gemini 3.1 Pro Preview) that approves or holds each APPEND and CORRECT in saved REVIEW proposals without seeing the extraction or the reference, and the DSTC2 replay.

5.2 Endpoints and the Variant-Tolerant Reference

The primary endpoint prespecified in the original study is positive-change completion: joint success of the action and the whole state on the 28 of the 48 main cases that require a new fact or the retirement of an obsolete version. A successful output must be valid, choose an allowed action, contain the required facts, omit forbidden and obsolete tuples, preserve protected facts, satisfy the required removals, and introduce no unlisted fact; provider failures and invalid outputs stay in the denominator. Strict correction completion applies the same criterion to the 11 correction-eligible cases. The reference-violation endpoint flags forbidden or extra writes and damage to protected facts across all 48 cases. Successful positive changes and reference-violating states are always reported together, so that withholding more operations is not mistaken for improving the task.

The reference matches tuples exactly, so an unlisted fact counts as a violation even when it restates a required fact in another form. To measure how much each conclusion depends on this convention, we re-scored every saved output under a *variant-tolerant reference*. A tuple counts as a required or protected fact when its relation and time match and, after Unicode NFC, case folding, and whitespace removal, its subjects are equal, both denote the participant, or one contains the other, and its values are equal or one contains the other; a participant label is never matched to another person’s label. Forbidden and obsolete tuples stay exact, so the second reference can only credit outputs. It accepts 38 distinct pairs in 10 cases, all listed in Online Appendix K, and the scorer reproduces every archived score of the exact reference. It is a mechanical convention, not a semantic judgment, and no human rated equivalence.

Table 5. Status of the reported analyses. “Prespecified” does not imply public preregistration.

Analysis	Inferential role
Controlled path versus REVIEW (original families)	Prespecified primary test; five-slot Holm accounting retained.
AGREE versus WITNESS	Planned secondary contrast; descriptive.
ALLOW, LITERAL, hold traces, exposure; gate on REVIEW proposals	Post hoc, on saved artifacts.
Variant audit, variant-tolerant re-scoring, decomposition	Post hoc and mechanical; no model call.
Later Gemini configurations; 21-configuration panel	Exploratory reuse of the same 48 cases.
Further sources and follow-up runs	Diagnostic; fixed before their outputs.

5.3 Paired Analysis

For configuration m and case i , the paired difference is $d_{mi} = \frac{1}{3} \sum_{r=1}^3 [Y_{mir}(\text{ctrl}) - Y_{mir}(\text{REVIEW})]$, where Y is the endpoint indicator for repetition r and ctrl denotes the controlled path; the effect estimate averages d_{mi} over cases. Uncertainty is reported as 95% percentile intervals from 5,000 case-cluster bootstrap draws that keep the repetitions, and for pooled estimates the configurations, of a case together (48 clusters for all-case endpoints, 28 for positive completion, and 11 for correction completion). The prespecified exact two-sided McNemar test counts a case as a success when it succeeds in at least two of three repetitions, and the original plan adjusted the five planned family tests with Holm’s method (Online Appendix A). Each original-protocol run made $48 \times 3 \times 6 = 864$ physical calls, of which each two-call pipeline used 288, without retries. Gemini 2.5 Flash used temperature 0.2, disabled thinking, and 2,048 output tokens; later configurations used their supported thinking settings, so the panel compares configurations as run rather than at equal compute (Online Appendix C). Counts pool configurations and repetitions, which are not independent replications of a case; distinct cases are reported alongside.

5.4 Confirmatory and Exploratory Analyses

Only the comparison of the controlled path with REVIEW was prespecified as a test; the study began as that test, and the replay analyses were added to explain its outcome (Table 5). Applying the gate to saved general-review proposals pairs each saved REVIEW proposal with the extraction from the same configuration, case, and repetition, applies one of the four rules, and executes the result with the frozen executor; we write REVIEW+WITNESS and so on. These conditions, two string-normalization controls, the variant audit, and the variant-tolerant re-scoring required no model call (Online Appendices I and K).

6 RESULTS

Component replay varies admission while holding the extraction and proposal fixed; complete-pipeline comparisons also let the generated artifacts change. We report the first before the second.

6.1 Withheld Corrections Originate in Extraction

On Gemini 2.5 Flash, the agreement gate withheld 11 required corrections that the other three rules admitted. Changing admission from AGREE to LITERAL or WITNESS increases completed corrections from 22 to 33 of 33 observations, while reference-violating states remain at 3 of 144 under every rule (Table 6). The 11 recovered corrections come from five of the 11 eligible cases, and eight are repetitions of self-repair cases. The paired case-mean gain is 33.3 percentage points (95%

Table 6. Comparisons on identical proposals in the seven Gemini configurations, in the order AGREE / ALLOW / LITERAL / WITNESS, under the exact reference. Strict correction uses 33 observations from 11 cases, and the reference-violation endpoint uses 144 observations from 48 cases. The H column reports exposed outputs and, in parentheses, distinct cases; by Proposition 4.1, no two rules can differ by more than this count. All counts combine three repetitions.

Configuration	Correction /33	Viol. /144	H outputs (cases)
Gemini 2.5 Flash	22 / 33 / 33 / 33	3 / 3 / 3 / 3	11 (5)
Gemini 2.5 Pro	31 / 31 / 31 / 31	3 / 9 / 4 / 3	6 (3)
Gemini 2.5 Flash-Lite	31 / 31 / 31 / 31	16 / 19 / 16 / 16	3 (1)
Gemini 3 Flash Preview	33 / 33 / 33 / 33	12 / 12 / 12 / 12	0 (0)
Gemini 3.1 Pro Preview	33 / 33 / 33 / 33	2 / 2 / 2 / 2	0 (0)
Gemini 3.5 Flash-Lite	32 / 32 / 32 / 32	7 / 12 / 7 / 7	5 (2)
Gemini 3.8 Flash	33 / 33 / 33 / 33	1 / 1 / 1 / 1	0 (0)

case-cluster interval 12.1 to 57.6), and the case-level McNemar test, with four discordant cases all favoring WITNESS, gives $p = .125$. Because ALLOW recovers the same corrections, releasing the hold is sufficient for recovery on these proposals.

The saved extractions show why the holds arose. In all 11 held observations (all three repetitions of CT45 and CT46, two of CT01 and CT48, and one of CT42), the first call returned an empty fact list for both readings and recorded the correction only as two affirmed witnesses whose tuples match the proposal exactly; in the 22 admitted observations, the corrected tuple appeared among the facts of both readings. The holds therefore follow from the absence of the tuple, not from the strictness of the comparison, and they did not protect against disagreeing readings, the situation that AGREE is designed for. This is the attribution confound: the final-state comparison planned before collection cannot tell whether the 33-point gain comes from target-linked evidence or from releasing these holds. The contrast set shows the same cause on new cases: AGREE held five supported corrections whose fact lists were empty in both readings, and all three releasing rules restored them; a sixth hold, in which one reading negates the new residence, is the only one of the 17 traced holds that reflects disagreement between readings (Online Appendix D).

A follow-up run tested this attribution directly (Online Appendix J). For each of the five cases, the repetition that had been held was re-extracted once under the original instruction and once under a revised instruction that also lists an affirmed correction among the reading's facts, and the unchanged AGREE gate was applied to the saved proposal. Under the original instruction, the corrected tuple was again absent from both fact lists in four cases (CT01 recovered spontaneously). Under the revised instruction, it appeared in both fact lists in all five, and AGREE admitted every correction: five completed against one, with no reference violation in either arm. On Gemini 3.8 Flash, whose proposals never reached H , the two instructions gave the same outcome on all 48 cases. Changing the extractor thus removes the holds whose release had been credited to the evidence checks, and changes nothing where no such hold existed.

6.2 Exposure Sets What a Rule Comparison Can Show

Gemini 3 Flash Preview, 3.1 Pro Preview, and 3.8 Flash produce no output that reaches H (Table 6). Their ties are instances of the opportunity confound, as is the 12-case boundary probe, in which no output reached H and every rule produced the same states (Online Appendix E). Across the 21 panel configurations, the saved gate decisions on controlled-path proposals show no exposed output on 12 configurations and 49 exposed outputs, from 12 cases, on the other nine (Online

Appendix G). By Proposition 4.1, no panel-wide comparison among the rules on these proposals can move an endpoint by more than 49 of 3,024 outputs, or 1.6 percentage points. Both evidence checks release 12 of the 49 holds, each completing a required correction (11 on Gemini 2.5 Flash and one on DeepSeek 3.2); LITERAL alone releases two more, both reference violations; and all three rules keep the other 35, all in cases that require no change, where the held state conforms to the reference. On most exposed outputs, the agreement gate thus withheld writes that the reference rejects, as designed.

Within the exposed outputs, a literal check supplies most of the protection that the evidence checks add. Relative to ALLOW, WITNESS avoids 14 reference-violating state observations and LITERAL 13, all on Gemini 2.5 Pro, 2.5 Flash-Lite, and 3.5 Flash-Lite and from three distinct cases, while both keep ALLOW’s strict-correction count on every configuration (Table 6). On the saved Qwen3 14B proposals, ALLOW produces 29 reference-violating states out of 144 and LITERAL and WITNESS 20 each, with strict correction at 33 of 33. Compared with the conservative end, WITNESS matches AGREE’s reference-violation count on every configuration while recovering the 11 Flash corrections.

6.3 Target-Linked Evidence Blocked Person and Polarity Errors

The one reference-violating state that the literal check admits on Gemini 2.5 Pro illustrates what the witness check adds (Figure 3). The utterance describes a son’s workplace, and the same proposal both appends the son’s workplace and corrects the participant’s stored workplace. Because the replacement value occurs in both readings, LITERAL, like ALLOW, admits the second update; the witness for the participant’s record is not affirmed, so WITNESS holds it and keeps the participant’s workplace. Target-linked witnesses can thus block a value that appears in every reading but belongs to another person. The contrast set supplies a second situation, a value that one reading negates (Section 6.1), and the third is an uncertain report about a travel destination with a request not to change it (CT11), which LITERAL admits and WITNESS holds on GLM 5 controlled-path proposals and on saved Qwen3 14B review proposals. Literal occurrence cannot tell whose fact a value is or whether a reading affirms it; binding the evidence to the target record can, and in these four observations it changed the executed state. Section 6.7 adds a fifth, a negated value among real recognition hypotheses.

6.4 The Reference Decides Whether the Gate Looks Protective

Applying the gate to saved REVIEW proposals more than halves reference-violating states under the exact reference, from 146 to 64 with WITNESS, at a cost of 40 positive changes (Table 7). The two bounds separate the parts of the gate: REVIEW+AGREE already reaches 64, and REVIEW+ALLOW 75. Counting decisions by operation locates the difference (Online Appendix I). All four rules treat APPEND identically and hold 108 of 1,087 proposed appends; these holds account for 71 of the 82 avoided reference violations and 36 of the 40 lost positive changes, and the other four losses come from invalid extractions. AGREE holds 23 of 762 proposed corrections, on five configurations; ALLOW releases all of them, LITERAL 13, and WITNESS 12, and each release shared by these rules completes a required correction. On these proposals, the gate acts mainly as an append policy.

The avoided violations are not all semantic errors. Comparing each held tuple with the states that conform to the reference for the same case, 13 of the 82 involve a forbidden tuple, and 52 of the remaining 69 restate a required fact with different spacing (6), subject label (29; e.g., “어머니 순자” (“mother Sunja”) for “순자”), or value form (17; e.g., “자전거 타기” (“riding a bicycle”) for “자전거”). In these 52 observations the required fact is absent with or without the gate, so the hold lowers the violation count without changing completion.

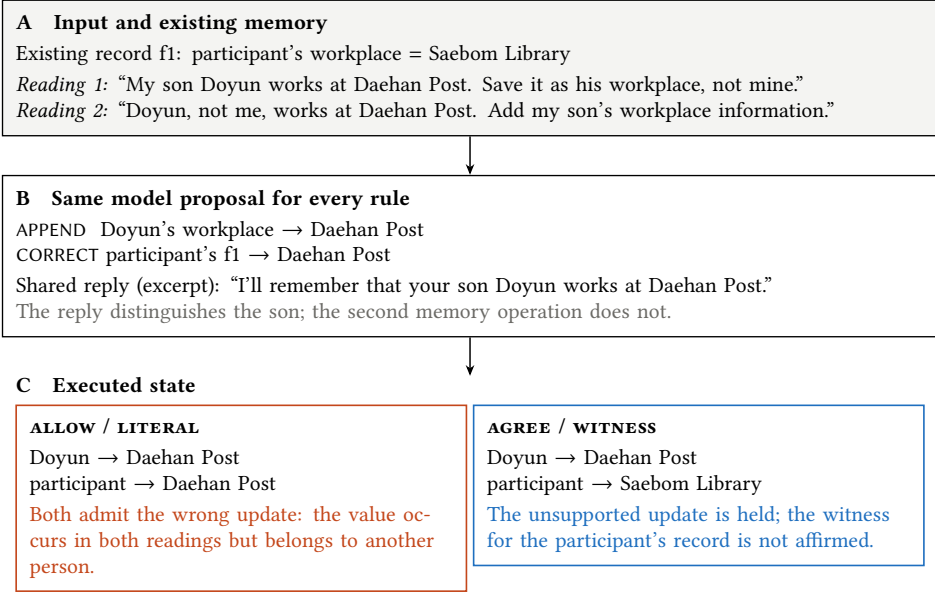


Fig. 3. Target binding changes the executed state. LITERAL accepts a workplace value that appears in both readings, whereas WITNESS rejects assigning the son's workplace to the participant. A correct-looking reply does not by itself guarantee a correct memory operation.

Table 7. The gate on saved REVIEW proposals under the exact and the variant-tolerant reference (21 configurations, 48 cases, three repetitions). Strict correction counts (690, 676, 688, 688, and 688 of 693) are the same under both references. The last column counts held decisions among 1,087 proposed APPEND and 762 proposed CORRECT operations.

Saved path	Viol. /3,024 ↓		Positive /1,764 ↑		Held APPEND / CORRECT
	Exact	Variant-tol.	Exact	Variant-tol.	
REVIEW	146	44	1,623	1,725	–
REVIEW + AGREE	64	14	1,571	1,621	108 / 23
REVIEW + ALLOW	75	25	1,583	1,633	108 / 0
REVIEW + LITERAL	65	15	1,583	1,633	108 / 10
REVIEW + WITNESS	64	14	1,583	1,633	108 / 11

The variant-tolerant reference measures how much this matters (Table 7). Under it, REVIEW produces 44 reference-violating states instead of 146 and 1,725 positive changes instead of 1,623: in 102 of its violating states, the only violation was a required fact written in another form. The gate then avoids 30 violations instead of 82 (paired difference 0.99 rather than 2.71 percentage points over 3,024 observations; case-cluster interval 0.40 to 1.69), and its cost rises from 40 to 92 positive changes (−5.22 points over 1,764, interval −8.22 to −2.49), because the appends it holds are now credited as required facts; whole-state conformity falls by 79 observations rather than 27. Whether the gate looks protective or costly on these proposals is decided by the reference: the specification confound at the level of a conclusion.

Comparisons among the rules do not move. On the 36 controlled-path outputs and the 23 review-proposal outputs on which the rules decide differently, the output-level difference between any two rules is the same under both references, so the differences between rules in Table 6 and among the gated rows of Table 7 are unchanged. This is the case that Proposition 4.1(ii) allows, because the released corrections carry exact values rather than variants. Comparing gating with not gating is different: its exposed set contains every output with a held append, and 52 of the 127 outputs on which REVIEW and REVIEW+WITNESS differ change their difference with the reference. Normalizing strings inside the gate, by contrast, changes no executed state across the 3,024 panel observations, because the recovery gap lies in facts absent from the extraction, not in their written form. Surface form matters inside the evidence checks on the public task-dialogue probe instead: among 10 eligible self-repairs, agreement on the focal task state is 10 for REVIEW, 7 for ALLOW, 4 for LITERAL and WITNESS, and 3 for AGREE, and case differences such as “Thursday” and “thursday” explain part of the gap (Online Appendix F).

6.5 The Complete-Pipeline Difference Is Two Opposite Terms

Under the exact reference, complete-pipeline differences split evenly across configurations. Among the 21 panel configurations, the controlled path increases positive-change completion relative to general review on 9, decreases it on 9, and ties on 3, and reference violations decrease on 12, increase on 7, and tie on 2; 11 configurations change direction across repetitions (Online Appendix G). The prespecified primary comparison on the original families found no completion advantage. On Gemini 2.5 Flash, REVIEW completes 82 of 84 positive-change observations and the controlled path 80 (difference -2.4 percentage points, 95% interval -6.0 to 0.0 ; McNemar $p = 1.00$); on Qwen3 14B the difference is -8.3 points (-23.8 to 6.0); and Llama 3.1 8B, outside the panel because its controlled path produced 68 invalid final outputs, completes 6 of 84 against 29 (-27.4 points, -44.0 to -10.7 ; Holm-adjusted $p = .0586$).

Replay explains the null result by splitting the difference exactly. Because REVIEW+WITNESS applies the controlled path’s extraction and gate to the review proposal, the complete-pipeline difference is the sum of a proposal term, the controlled path minus REVIEW+WITNESS, in which only the proposal differs, and a gate term, REVIEW+WITNESS minus REVIEW, which Section 6.4 splits by operation (Table 8, Figure 4). Under the exact reference, the controlled path’s proposals add 67 reference violations and 17 positive changes relative to review proposals under the same gate (violations $+2.22$ points, interval 1.03 to 3.67), and the gate removes 82 violations and 40 positive changes; the two terms nearly cancel, leaving -15 and -23 . General review also completes all 33 strict corrections on Gemini 2.5 Flash, so the fixed-proposal gain has nothing to recover there. Under the variant-tolerant reference, the proposal term is $+40$ violations and $+44$ positive changes and the gate term -30 and -92 ; the controlled path then completes fewer positive changes on 13 configurations and more on one (pooled -2.72 points, -4.59 to -1.02), and violations decrease on 6 configurations and increase on 6. The direction of the complete-pipeline comparison thus depends on the reference, and the decomposition shows why: the gate’s append holds, not the extraction-informed proposals, carry the completion cost.

6.6 A Learned Verifier on the Same Saved Proposals

The protocol also accepts a learned rule. The verifier of Section 5.1 [50] replaces the whole gate on one saved REVIEW proposal per case for three configurations; for Gemini 2.5 Flash, the selected repetitions include a held repetition of each case of Section 6.1 (Online Appendix J). On Gemini 2.5 Flash, it reproduces REVIEW+WITNESS on every endpoint (11 of 11 corrections, 27 of 28 positive changes, one reference violation in 48). Against REVIEW+AGREE it completes five more corrections, which a final-state comparison would credit to the verifier, although they are the

Table 8. Exact decomposition of the complete-pipeline difference, controlled path minus REVIEW, pooled over 21 configurations, 48 cases, and three repetitions (counts of observations; reference violations over 3,024, positive changes over 1,764). The proposal term compares two proposals under the same extraction and WITNESS gate; the gate terms attribute each paired change to the held operation, and no observation changed both.

Term	Reference violations		Positive changes	
	Exact	Variant-tol.	Exact	Variant-tol.
Proposal: controlled path – REVIEW+WITNESS	+67	+40	+17	+44
Gate: held APPEND	–71	–19	–36	–88
Gate: held CORRECT	–11	–11	0	0
Gate: invalid extraction	0	0	–4	–4
Complete pipeline: controlled path – REVIEW	–15	+10	–23	–48

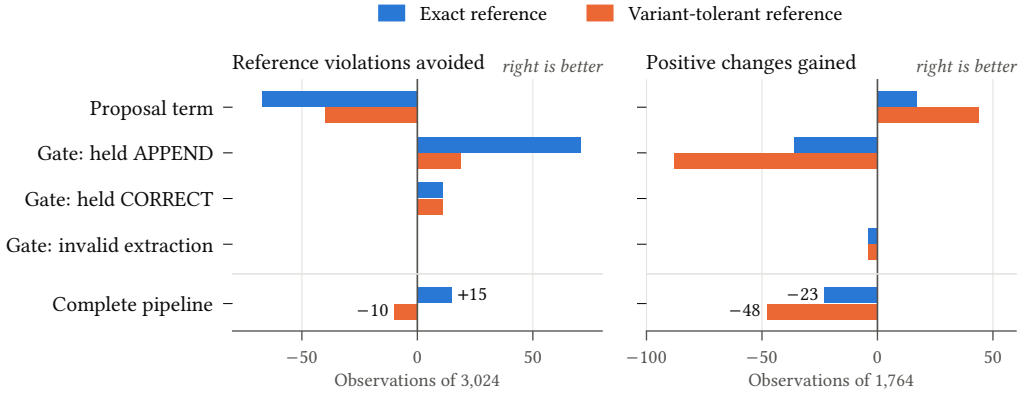


Fig. 4. The complete-pipeline difference as two opposite terms (Table 8), oriented so that improvement is to the right. The controlled path’s proposals add violations and completions; the gate, mostly through held appends, removes both. Under the variant-tolerant reference, the gate’s completion cost dominates and the net comparison reverses its sign for violations.

extraction-induced holds of Section 6.1 that any rule ignoring the extracted fact lists releases. Its other differences from REVIEW+WITNESS lie outside H . On Gemini 3.8 Flash, it admits the required append that Youngsook learned from Master Lee (CT16), which the shared append hold withheld because the extraction named the subject “동생 영숙” (“sibling Youngsook”) rather than “영숙”. On Qwen3 14B, it holds two writes that AGREE had admitted and that no evidence check therefore inspected, a shop-name correction that the participant was unsure of and asked not to make (CT12) and an append that assigns the sibling’s teacher to the participant (CT16), leaving one reference violation against three. Each paired interval against REVIEW+WITNESS includes zero, so these observations locate where a learned rule acts, on appends and on operations outside H , rather than rank it. The verifier made 144 calls for 144 proposals, with a mean latency of 4.39 seconds and about 850 tokens per call; replaying a deterministic rule makes none.

Table 9. Admission rules on real recognition hypotheses (DSTC2, Gemini 3.1 Pro Preview, one run). Goal conformity requires the exact food, area, and price-range goal; required changes count the 36 turns whose goal changes; a new wrong write introduces a value absent from the official goal.

Rule	Goal conformity /48 ↑	Required changes /36 ↑	New wrong writes /48 ↓
Ungated proposal	33	23	3
ALLOW	33	23	3
AGREE	18	6	0
LITERAL	17	6	1
WITNESS	18	6	0

6.7 Real Speech-Recognition Hypotheses

Unlike the written readings of the other sources, the DSTC2 source supplies hypotheses that a speech recognizer produced [13]. One user turn was selected from each of 48 test dialogues (41 callers) by a hash-ranked rule fixed before any output: 24 goal revisions, 12 denials without an explicit replacement, and 12 turns with recognition errors, a stratum selected with the labels and reported separately. The readings are the 433 distinct non-empty hypotheses of the live 10-best lists, in rank order and without likelihood weights. The initial memory is the previous official goal for food, area, and price range; the reference is the current official goal, derived from human-reviewed semantic labels and matched exactly; and the current transcript and labels were withheld from the model. The two-call pipeline, with the three slots in place of personal relations and the gate unchanged, ran once on Gemini 3.1 Pro Preview, all 48 outputs valid (Table 9); a partial Gemini 3.8 Flash run is reported in Online Appendix J.

Exposure is high on this source: AGREE held a proposed correction in 20 of the 48 outputs, against at most 11 of 144 for any main-source configuration. The trace also returns a different attribution. None of the 20 holds shows the pattern of Section 6.1, affirmed witnesses in every reading with the tuple missing from the fact lists; in ten, different hypotheses yield different extracted values, and in the other ten the value is supported by only some hypotheses or omitted from some. Releasing every hold (ALLOW) completes 17 required changes and introduces 3 wrong values. Neither evidence check releases any of the 17, because both require the value in every hypothesis, and LITERAL releases one hold, a wrong write. Relative to ALLOW, WITNESS lowers goal conformity by 31.25 percentage points (95% caller-cluster interval 16.3 to 45.8) and required-change completion by 47.2 points (32.3 to 62.9), and it lowers turns with a new wrong write by 6.25 points (0.0 to 13.5).

Two turns show the trade-off. In DSTC2_45, the caller says “i dont want chinese i want thai food”; the stored goal is already thai, but every hypothesis contains “chinese,” and the proposal changes the goal to chinese. LITERAL admits the change because the value occurs in every hypothesis, whereas WITNESS, like AGREE, holds it for lack of affirmed, target-linked witnesses: the polarity error of Section 6.3 on a real recognizer. In DSTC2_07, “how about vietnamese food” appears in some hypotheses as “how about you need food”; ALLOW admits the correct change, and the three strict rules hold it. On this source, agreement does what it was designed for, keeping values on which the hypotheses disagree out of memory, and replay prices it: requiring support in every reading suits written alternatives better than n-best lists.

7 DISCUSSION

7.1 What Fixed-Proposal Evaluation Adds

Fixed-proposal evaluation connects executed-state scores to recorded admission decisions. Within an unchanged extraction, proposal, executor, and reference, it shows which required corrections are withheld and which admitted operations produce reference violations, and exposure states how large any difference between rules could be. The three confounds are not new as phenomena: stage-level oracle analysis [15] and lists of equivalent values [46] address versions of the first two in state tracking. What replay adds is a way to separate them on the same saved artifacts and to test each separation. An ALLOW bound and the saved extraction attributed the recovered corrections, a revised extractor confirmed the attribution, a second reference showed which conclusions the exact reference drives, and on recognizer hypotheses the same trace returned a different cause. The audit in Table 1 indicates why this matters for memory writes: evaluations that score writes only through downstream answers cannot report exposure or separate a missed write from a withheld one. Running a control as a separately generated pipeline would also mix in generation variance, as the 11 configurations whose complete-pipeline direction changed across repetitions show.

7.2 Reporting a Memory-Write Comparison

The analyses suggest four reporting steps. First, report exposure and the bound it implies before interpreting a tie or a small difference. Second, replay the compared rule with an ALLOW bound on the same proposals and inspect the stage that created the holds it releases. Third, report completed changes and reference violations together, under the exact reference and a stated variant-tolerant one, and say which conclusions differ. Fourth, decompose a complete-pipeline difference into proposal and gate terms before crediting either. Once outputs are saved, none of these steps requires a model call.

7.3 Replay Cost and Pipeline Resources

Separating admission from generation makes deterministic policy changes inexpensive to test: replaying a rule on saved proposals requires no model call, whereas evaluating a different policy inside a sampled review call requires regenerating the call, which for the original Flash REVIEW pipeline meant 288 calls, 512,062 input and 54,918 output tokens, and 431.9 seconds of summed provider latency. Deploying the gate costs more than replaying it. The hybrid evaluated here combines the saved REVIEW draft and proposal with the controlled path's extraction, so a standalone implementation needs three generation stages rather than two; for Gemini 2.5 Flash, the summed stage times average 3.00 seconds for REVIEW and 4.92 seconds for the hybrid. Because the extraction contains one witness for each pair of a stored record and a reading, larger memories would require scoping it to the records that a proposal targets.

8 SCOPE AND FUTURE WORK

All reported counts are exact and reproducible by replay; what they generalize to is bounded by the sources, the reference, and the pipeline. The evidence comes from 48 authored Korean cases with written alternative readings, run on 21 model configurations with three repetitions, and from 48 DSTC2 turns with recognizer hypotheses in an English task domain; the follow-up runs are single runs on few units. The rule differences rest on few distinct cases: five for the recovered corrections, three for the literal check's protection, and four observations from three situations for the witness-only holds. All replays use our own two-call extraction and proposal pipeline, so whether externally developed memory writers produce extraction-induced holds is untested, and how often each confound arises in other systems is a prevalence question that these selected

sources cannot answer. The original primary comparison is the only confirmatory result, and it is null for completion (Table 5).

The reference remains the main constraint. The main-source cases and references were written by the authors with AI assistance, and no human judged equivalence, so no agreement statistic is available. The variant-tolerant re-scoring shows how far the exact convention drives each conclusion, but it is itself mechanical: it may accept a variant that a person would reject or miss one that needs translation. Only the DSTC2 reference derives from human-reviewed labels. The polarity of a witness is a model output checked for structure and linkage rather than established entailment, and the study makes no claim about user-perceived usefulness, semantic safety, a validated deployment policy, or downstream conversational quality.

Three comparisons follow directly. On recognizer output, admission that weights hypotheses by likelihood or requires a majority can be replayed on the saved DSTC2 extractions and proposals without new model calls and is the next rule to compare. The revised extraction instruction should be fixed on development cases and run across configurations, with admission rules replayed within each extractor. Admission rules for appended facts, including the learned verifier at scale, should be compared against human judgments of equivalence with reported agreement. The Korean memory-operation data of Bae et al. [4] would supply a human-labeled reference, but the public release we inspected lacks the original operation labels and splits. The study does not evaluate serving latency, recognition accuracy, subsequent retrieval, or longitudinal interaction.

9 CONCLUSION

Specification-based replay makes memory-write outcomes traceable to admission decisions on fixed generated artifacts and separates three confounds that a final-state score cannot distinguish. The correction gain of evidence checks over agreement was the release of holds created by extraction, and a revised extraction instruction removed them; most ties reflected zero exposure; and the apparent safety of a gate on general-review proposals depended on the reference, whereas no comparison among rules on fixed proposals did. The same replay explained a prespecified null result as two opposite terms, proposals that added both completions and violations and a gate that traded them back, and on real recognition hypotheses it traced holds to disagreement between hypotheses, where strict admission withheld 17 correct changes to prevent 3 wrong writes. Within held corrections, target binding changed the executed state in five observations involving another person's value, a negated value, or a value that the participant asked not to change. Before crediting an admission rule, replay it on fixed proposals with an ALLOW bound, report its exposure, score it under two references, and decompose any complete-pipeline difference; none of these steps requires a new model call.

DATA, CODE, AND AI USE

The archive at <https://anonymous.4open.science/r/memory-admission-9E37/> (artifact version paper-artifact-v6) contains the Korean test cases, references, prompts, raw outputs and failures, executed states, the evaluator, and the analysis code, including the variant-tolerant re-scoring, with provenance hashes; for DSTC2, which is used under its own terms and not redistributed, it contains the selected turn identifiers and the code that reconstructs the inputs. The new experiments involved no human participants. ChatGPT and Codex assisted with software development, test-case drafting, literature discovery, analysis code, and manuscript editing; the authors reviewed all such content and take responsibility for it. English examples are translations of the Korean inputs that define the experiments.

REFERENCES

- [1] Vaibhav Adlakha, Parishad BehnamGhader, Xing Han Lu, Nicholas Meade, and Siva Reddy. 2024. Evaluating Correctness and Faithfulness of Instruction-Following Models for Question Answering. *Transactions of the Association for Computational Linguistics* 12 (2024), 681–699. https://doi.org/10.1162/tacl_a_00667
- [2] Anirudh Ajith, Chris Pan, Mengzhou Xia, Ameet Deshpande, and Karthik Narasimhan. 2024. InstructEval: Systematic Evaluation of Instruction Selection Methods. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, 4336–4350. <https://doi.org/10.18653/v1/2024.findings-naacl.270>
- [3] Jacob Andreas, John Bufe, David Burkett, Charles Chen, Josh Clausman, Jean Crawford, Kate Crim, Jordan DeLoach, Leah Dörner, Jason Eisner, et al. 2020. Task-Oriented Dialogue as Dataflow Synthesis. *Transactions of the Association for Computational Linguistics* 8 (2020), 556–571. https://doi.org/10.1162/tacl_a_00333
- [4] Sanghwan Bae, Donghyun Kwak, Soyoung Kang, Min Young Lee, Sungdong Kim, Yuin Jeong, Hyeri Kim, Sang-Woo Lee, Woomyoung Park, and Nako Sung. 2022. Keep Me Updated! Memory Management in Long-term Conversations. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 3769–3787. <https://doi.org/10.18653/v1/2022.findings-emnlp.276>
- [5] Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. 2026. STALE: Can LLM Agents Know When Their Memories Are No Longer Valid? <https://doi.org/10.48550/arXiv.2605.06527> arXiv:2605.06527
- [6] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems*, Vol. 37. 130185–130213. <https://doi.org/10.52202/079017-4136>
- [7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. <https://doi.org/10.48550/arXiv.2504.19413> arXiv:2504.19413
- [8] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *Advances in Neural Information Processing Systems*, Vol. 38. <https://openreview.net/forum?id=i7J6t2wtV>
- [9] Yijiang River Dong, Tiancheng Hu, Zheng Hui, Caiqi Zhang, Ivan Vulić, Andreea Bobu, and Nigel Collier. 2026. Value of Information: A Framework for Human-Agent Communication. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 42879–42896. <https://doi.org/10.18653/v1/2026.acl-long.1987>
- [10] Felix Gervits, Gordon Briggs, Antonio Roque, Genki A. Kadamatsu, Dean Thurston, Matthias Scheutz, and Matthew Marge. 2021. Decision-Theoretic Question Generation for Situated Reference Resolution: An Empirical Study and Computational Model. In *Proceedings of the 2021 International Conference on Multimodal Interaction*. ACM, 150–158. <https://doi.org/10.1145/3462244.3479925>
- [11] Shengyue Guan, Jindong Wang, Jiang Bian, Bin Zhu, Jian-Guang Lou, and Haoyi Xiong. 2026. Evaluating LLM-based Agents for Multi-turn Conversations: A Survey. *ACM Transactions on Intelligent Systems and Technology* 17, 4, Article 85 (2026), 40 pages. <https://doi.org/10.1145/3793671>
- [12] Jinyu Guo, Zhaokun Wang, Jingwen Pu, Wenhong Tian, Guiduo Duan, and Guangchun Luo. 2025. Multi-Perspective Dialogue Non-Quota Selection with loss monitoring for dialogue state tracking. *Expert Systems with Applications* 283 (2025), 127516. <https://doi.org/10.1016/j.eswa.2025.127516>
- [13] Matthew Henderson, Blaise Thomson, and Jason D. Williams. 2014. The Second Dialog State Tracking Challenge. In *Proceedings of the 15th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*. Association for Computational Linguistics, Philadelphia, PA, USA, 263–272. <https://doi.org/10.3115/v1/W14-4337>
- [14] Or Honovich, Roei Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kukliansy, Vered Cohen, Thomas Scialom, Idan Szepkter, Avinatan Hassidim, and Yossi Matias. 2022. TRUE: Re-evaluating Factual Consistency Evaluation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Seattle, United States, 3905–3920. <https://doi.org/10.18653/v1/2022.naacl-main.287>
- [15] Sungdong Kim, Sohee Yang, Gyuwan Kim, and Sang-Woo Lee. 2020. Efficient Dialogue State Tracking by Selectively Overwriting Memory. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 567–582. <https://doi.org/10.18653/v1/2020.acl-main.53>
- [16] Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. 2025. Hello Again! LLM-powered Personalized Agent for Long-term Dialogue. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 5259–5276. <https://doi.org/10.18653/v1/2025.naacl-long.272>
- [17] Hongjun Liu and Chen Zhao. 2026. Memory Is a Derivation: The Distributed-Evidence Paradox in Long-Term Agents. <https://doi.org/10.48550/arXiv.2609.36130> arXiv:2609.36130
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational*

- Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638
- [19] Zhanghui Liu, Chaoxiong Zhou, Yuzhong Chen, and Zeping Zheng. 2026. Few-shot dialogue state tracking via context ranking and dynamic contrastive decoding. *Expert Systems with Applications* 300 (2026), 130197. <https://doi.org/10.1016/j.eswa.2025.130197>
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. <https://doi.org/10.52202/075280-2019>
- [21] Chris Madge, Matthew Purver, and Massimo Poesio. 2025. Referential ambiguity and clarification requests: comparing human and LLM behaviour. In *Proceedings of the Eighth Workshop on Computational Models of Reference, Anaphora and Coreference*. Association for Computational Linguistics, 1–11. <https://doi.org/10.18653/v1/2025.crac-1.1>
- [22] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 13851–13870. <https://doi.org/10.18653/v1/2024.acl-long.747>
- [23] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. <https://doi.org/10.48550/arXiv.2310.08560> arXiv:2310.08560
- [24] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. 2025. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. In *International Conference on Learning Representations*, Vol. 2025. 91851–91885. https://proceedings.iclr.cc/paper_files/paper/2025/hash/e56f394bbd4f0ec81393d767caa5a31b-Abstract-Conference.html
- [25] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–22. <https://doi.org/10.1145/3586183.3606763>
- [26] Sudha Rao and Hal Daumé III. 2018. Learning to Ask Good Questions: Ranking Clarification Questions using Neural Expected Value of Perfect Information. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2737–2746. <https://doi.org/10.18653/v1/P18-1255>
- [27] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. Measuring Attribution in Natural Language Generation Models. *Computational Linguistics* 49, 4 (2023), 777–840. https://doi.org/10.1162/coli_a_00486
- [28] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. <https://doi.org/10.48550/arXiv.2501.13956> arXiv:2501.13956
- [29] Wout Schellaert, Fernando Martínez-Plumed, and José Hernández-Orallo. 2025. Analysing the Predictability of Language Model Performance. *ACM Transactions on Intelligent Systems and Technology* 16, 2, Article 30 (2025), 26 pages. <https://doi.org/10.1145/3706118>
- [30] Ivan Sekulić, Mohammad Aliannejadi, and Fabio Crestani. 2024. Analysing Utterances in LLM-Based User Simulation for Conversational Search. *ACM Transactions on Intelligent Systems and Technology* 15, 3, Article 62 (2024), 22 pages. <https://doi.org/10.1145/3650041>
- [31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. <https://doi.org/10.52202/075280-0377>
- [32] Leila Tavakoli, Johanne R. Trippas, Hamed Zamani, Falk Scholer, and Mark Sanderson. 2025. Online and Offline Evaluation in Search Clarification. *ACM Transactions on Information Systems* 43, 1, Article 2 (2025), 30 pages. <https://doi.org/10.1145/3681786>
- [33] Dechuan Teng, Zheng Yan, and Wanxiang Che. 2026. Program of actions (PoA): Interactive semantic action parser as more powerful dialogue state tracker through code generation. *Expert Systems with Applications* 301 (2026), 130481. <https://doi.org/10.1016/j.eswa.2025.130481>
- [34] Alberto Testoni and Raquel Fernández. 2024. Asking the Right Question at the Right Time: Human and Model Uncertainty Guidance to Ask Clarification Questions. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 258–275. <https://doi.org/10.18653/v1/2024.eacl-long.16>
- [35] Xin Tian, Liankai Huang, Yingzhan Lin, Siqi Bao, Huang He, Yunyi Yang, Hua Wu, Fan Wang, and Shuqi Sun. 2021. Amendable Generation for Dialogue State Tracking. In *Proceedings of the 3rd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, 80–92. <https://doi.org/10.18653/v1/2021.nlp4convai-1.8>

- [36] Bo-Hsiang Tseng, Shruti Bhargava, Jiarui Lu, Joel Ruben Antony Moniz, Dhivya Piraviperumal, Lin Li, and Hong Yu. 2021. CREAD: Combined Resolution of Ellipses and Anaphora in Dialogues. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 3390–3406. <https://doi.org/10.18653/v1/2021.naacl-main.265>
- [37] Qingyue Wang, Yanhe Fu, Yanan Cao, Shuai Wang, Zhiliang Tian, and Liang Ding. 2025. Recursively summarizing enables long-term dialogue memory in large language models. *Neurocomputing* 639 (2025), 130193. <https://doi.org/10.1016/j.neucom.2025.130193>
- [38] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=1PL1NIMMrw>
- [39] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*, Vol. 2025. 86809–86836. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html
- [40] Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. 2024. Knowledge Conflicts for LLMs: A Survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 8541–8565. <https://doi.org/10.18653/v1/2024.emnlp-main.486>
- [41] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA, 17577–17604. <https://doi.org/10.52202/085713-0593>
- [42] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2026. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12805–12825. <https://doi.org/10.18653/v1/2026.acl-long.583>
- [43] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR)*. 9965–10017. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html
- [44] Fanghua Ye, Jarana Manotumruksa, and Emine Yilmaz. 2022. MultiWOZ 2.4: A Multi-Domain Task-Oriented Dialogue Dataset with Essential Annotation Corrections to Improve State Tracking Evaluation. In *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*. Association for Computational Linguistics, Edinburgh, UK, 351–360. <https://doi.org/10.18653/v1/2022.sigdial-1.34>
- [45] Steve Young, Milica Gašić, Blaise Thomson, and Jason D. Williams. 2013. POMDP-Based Statistical Spoken Dialog Systems: A Review. *Proc. IEEE* 101, 5 (2013), 1160–1179. <https://doi.org/10.1109/JPROC.2012.2225812>
- [46] Xiaoxue Zang, Abhinav Rastogi, Srinivas Sunkara, Raghav Gupta, Jianguo Zhang, and Jindong Chen. 2020. MultiWOZ 2.2: A Dialogue Dataset with Additional Annotation Corrections and State Tracking Baselines. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, 109–117. <https://doi.org/10.18653/v1/2020.nlp4convai-1.13>
- [47] Guilin Zhang, Wei Jiang, Xiejiaohan Wang, Aisha Behr, Kai Zhao, Jeffrey Friedman, Xu Chu, and Amine Anoun. 2026. Adaptive Memory Admission Control for LLM Agents. <https://doi.org/10.48550/arXiv.2603.04549> arXiv:2603.04549
- [48] Michael JQ Zhang and Eunsol Choi. 2025. Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 5541–5558. <https://doi.org/10.18653/v1/2025.findings-naacl.306>
- [49] Yan Zhang and Shibo Li. 2026. ConsistencyGate: Preventing Memory Contamination in LLM Agents via Self-Consistency Admission Control. <https://doi.org/10.48550/arXiv.2607.22962> arXiv:2607.22962
- [50] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46595–46623. <https://doi.org/10.52202/075280-2020>
- [51] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 17 (2024), 19724–19731. <https://doi.org/10.1609/aaai.v38i17.29946>